

05b — Client Side Caching

Key Concepts

- **Client side caching** — browser stores response data locally based on server-sent HTTP headers
- **Cache-Control** — HTTP header that tells the browser how to cache a response
- **ETag** — a fingerprint (hash) of the response; used to check if content changed
- **304 Not Modified** — server response meaning "your cache is still valid, use it"
- **Revalidation** — browser asks server "has this changed?" before using cached data

How It Works

```
First request:  Client → Server → returns data + Cache-Control / ETag headers
Second request: Client checks local cache → serves from cache (no network call)

ETag flow:
  Browser sends: If-None-Match: "abc123"
  Server checks: data changed? → 200 + new data | unchanged? → 304 (no body)
```

Cache-Control Header Values

Value	Meaning	Use Case
max-age=3600	Cache for 3600 seconds	Static assets
no-cache	Always revalidate with server before using	Frequently updated data
no-store	Never cache at all	Banking, sensitive pages
public	Anyone can cache (CDN + browser)	Same data for all users
private	Only browser can cache	Personalized / user-specific data

ETag — Who Does What

Who	Responsibility
Server framework (ASP.NET, Express)	Generates and sends ETag header automatically
Browser	Stores ETag, sends If-None-Match on next request automatically
Frontend developer	Nothing — fully transparent
Backend developer	Can customize ETag logic (e.g., based on updated_at column)

Why ETag is still efficient even with a DB check:

- DB check = cheap (`SELECT updated_at` only)
- If unchanged → 304, no body sent, no serialization
- Saves network bandwidth on large responses

public vs private — Key Rule

`public` = same data for everyone → safe for CDN caching `private` = user-specific data → only safe in the user's own browser

Industry Examples

- **Google Fonts** — `max-age` of 1 year; browser never re-downloads fonts
 - **GitHub API** — ETags on responses; clients skip full fetch when nothing changed
 - **Netflix** — `public` cache on static assets (JS, CSS, images) at CDN level
 - **Facebook / Instagram** — `private` on feeds; personalized per user, CDN caching would leak data
-

Interview Questions

Q: What is client side caching?

Browser stores response data locally based on Cache-Control headers sent by the server. Saves bandwidth and reduces server load.

Q: What is the difference between no-cache and no-store?

`no-cache` = cache it but always revalidate with server before using. `no-store` = never cache at all (used for sensitive data like banking).

Q: What is an ETag and how does it work?

A fingerprint of the response. Browser sends it back on next request via `If-None-Match`. If unchanged, server returns 304 with no body — saves bandwidth.

Q: What is the difference between public and private in Cache-Control?

`public` = any cache (CDN, browser) can store it — used for shared content. `private` = only the user's browser — used for personalized data like social feeds.

Q: Does the developer have to write ETag logic?

No — server frameworks generate ETags automatically and browsers handle sending them. Developers only intervene for custom ETag strategies.